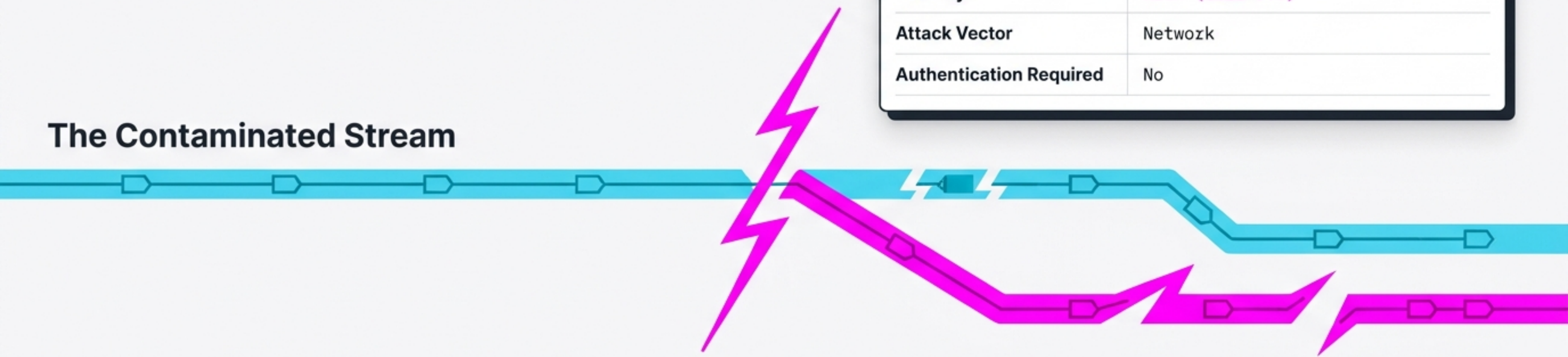


CVE-2026-33128: Manipulating the Server-Sent Event Stream in h3

Vulnerability Anatomy, Impact, and Mitigation

Vitals	
Product	h3 Framework (h3js)
Vulnerability Type	Server-Sent Events (SSE) Injection
Root Cause	CRLF Injection (CWE-93)
Severity	HIGH (CVSS 7.5)
Attack Vector	Network
Authentication Required	No

The Contaminated Stream



Server-Sent Events establish a continuous, trusted, one-way pipeline.

The Framework



h3 Framework

A lightweight HTTP framework powering:

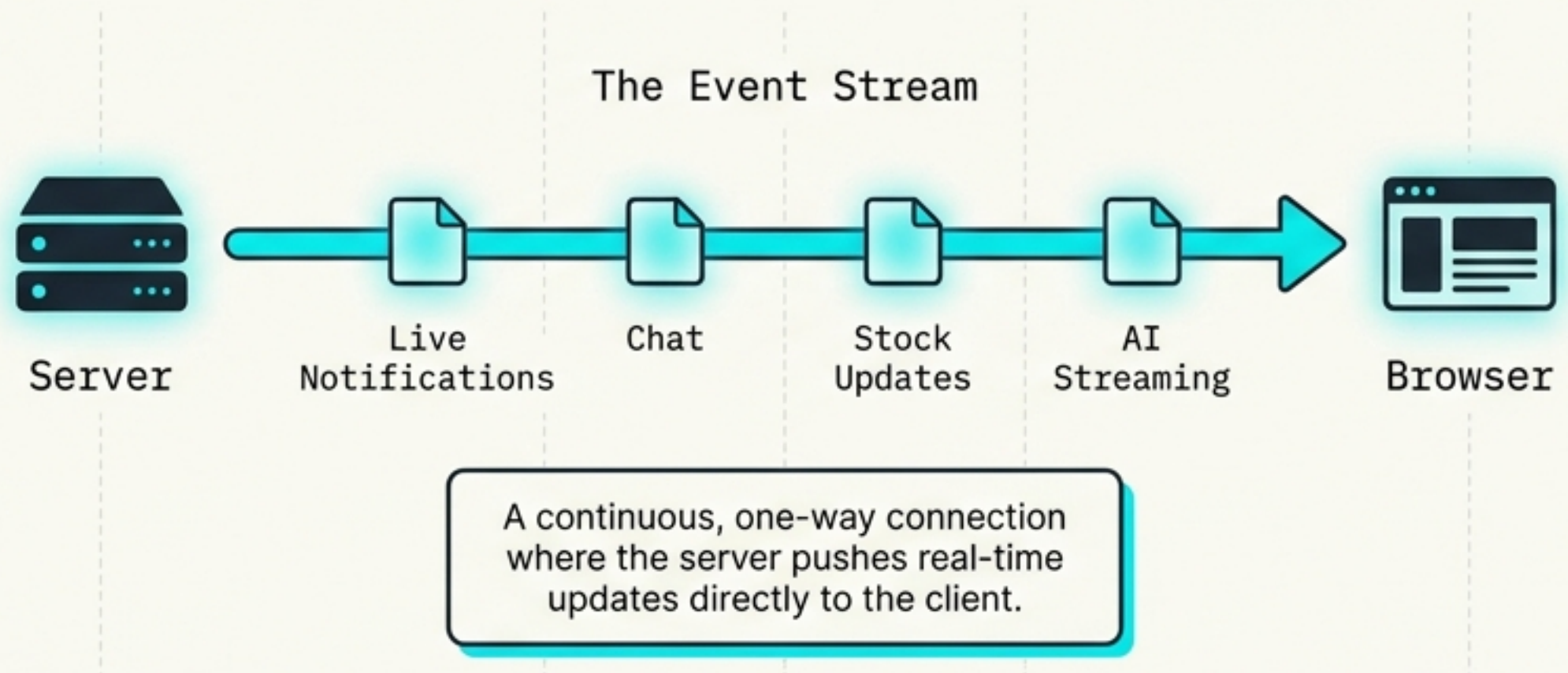
Nuxt.js

Nitro API Servers

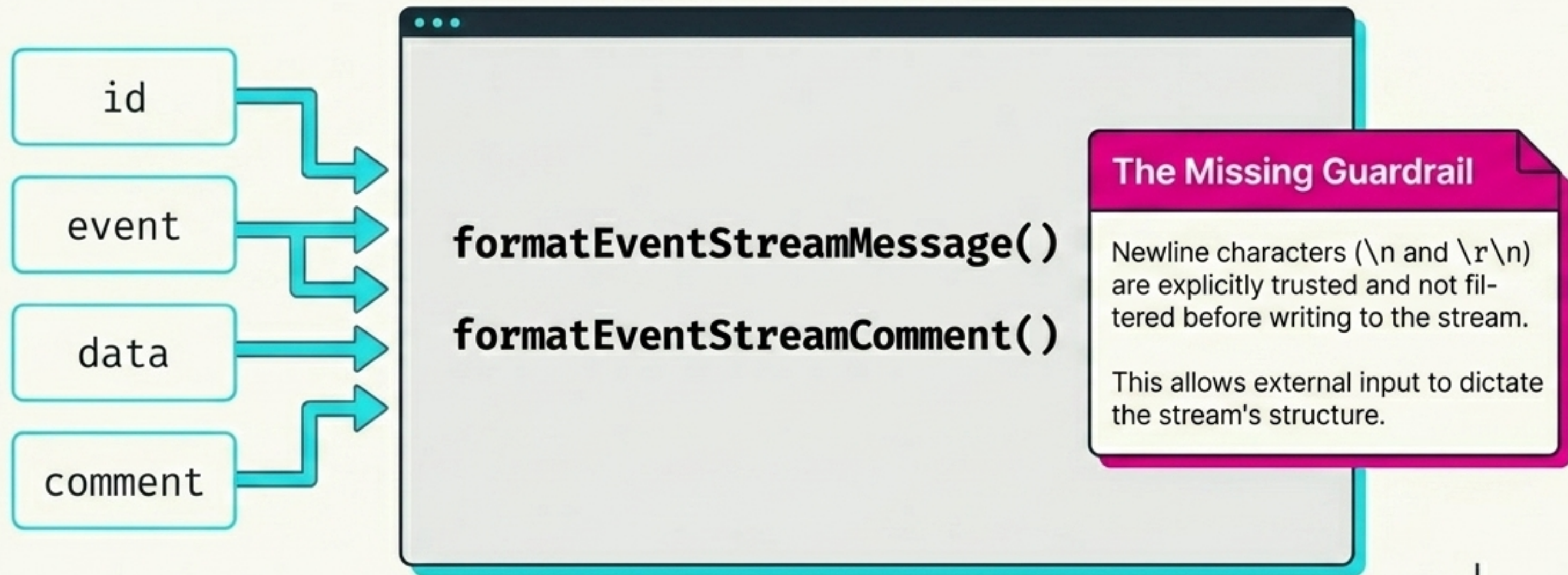
Real-Time Apps

Web Services

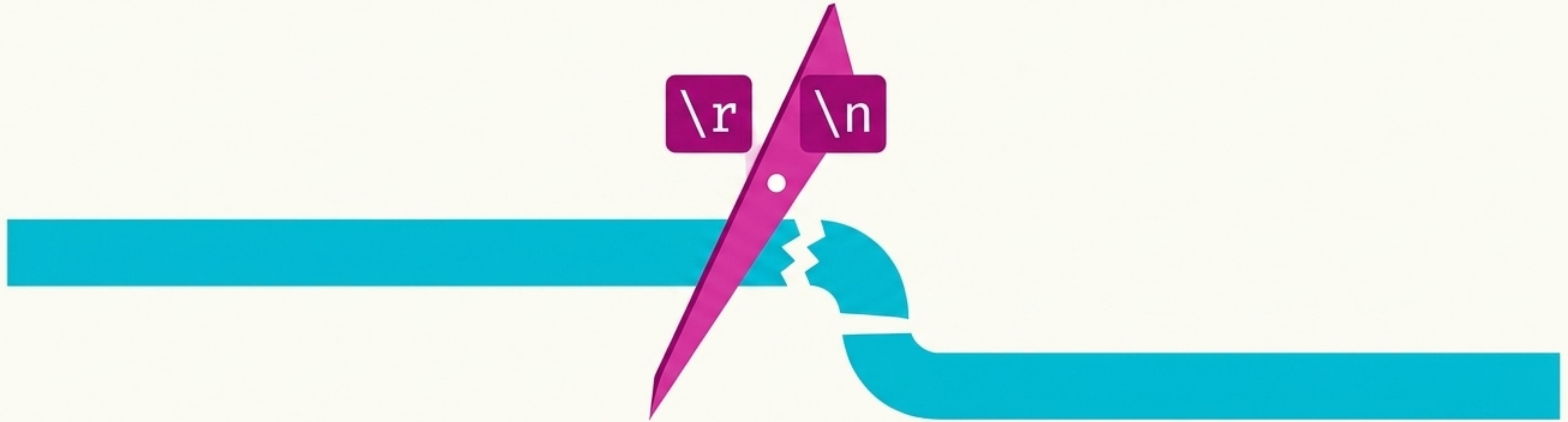
The Protocol (SSE)



User-controlled inputs bypass sanitization and enter the stream directly.



Carriage Returns and Line Feeds act as structural wedges within the stream.



Attackers inject line breaks not to crash the application, but to rewrite the server's responses. By forcing a line break, the remaining payload is validated by the protocol as a fresh, server-authenticated command.

A single injected newline tricks the server into generating multiple distinct events.

Legitimate Event Stream

User Input:

Hello



```
event: chat
data: Hello
```

One Input -> One Event Generated

Corrupted Event Stream

Malicious Input:

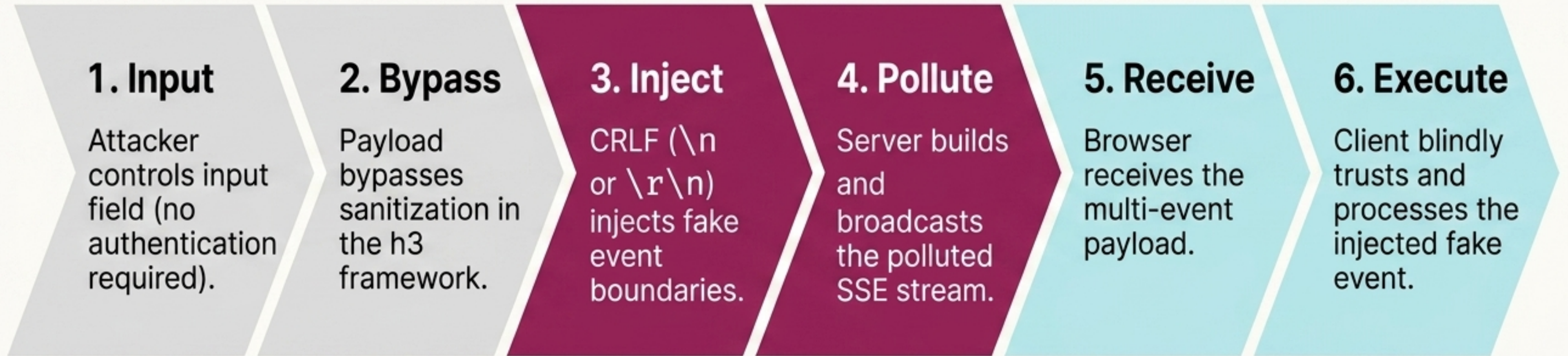
```
Hello \n
event: admin \n
data: Unauthorized Action
```



```
event: chat
data: Hello
event: admin
data: Unauthorized Action
```

One Input -> Multiple Events Generated

The vulnerability lifecycle travels from public input to client execution.



The flaw compromises data integrity without requiring authentication or complex conditions.



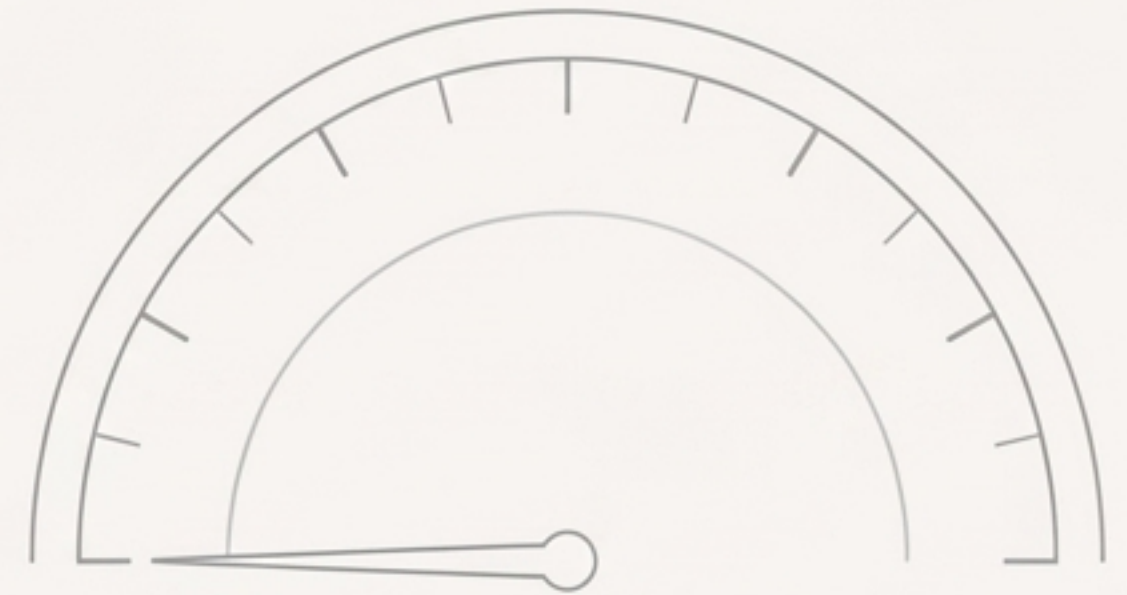
Confidentiality: LOW

Limited data exposure.



Integrity: HIGH

Complete event manipulation and client data tampering.



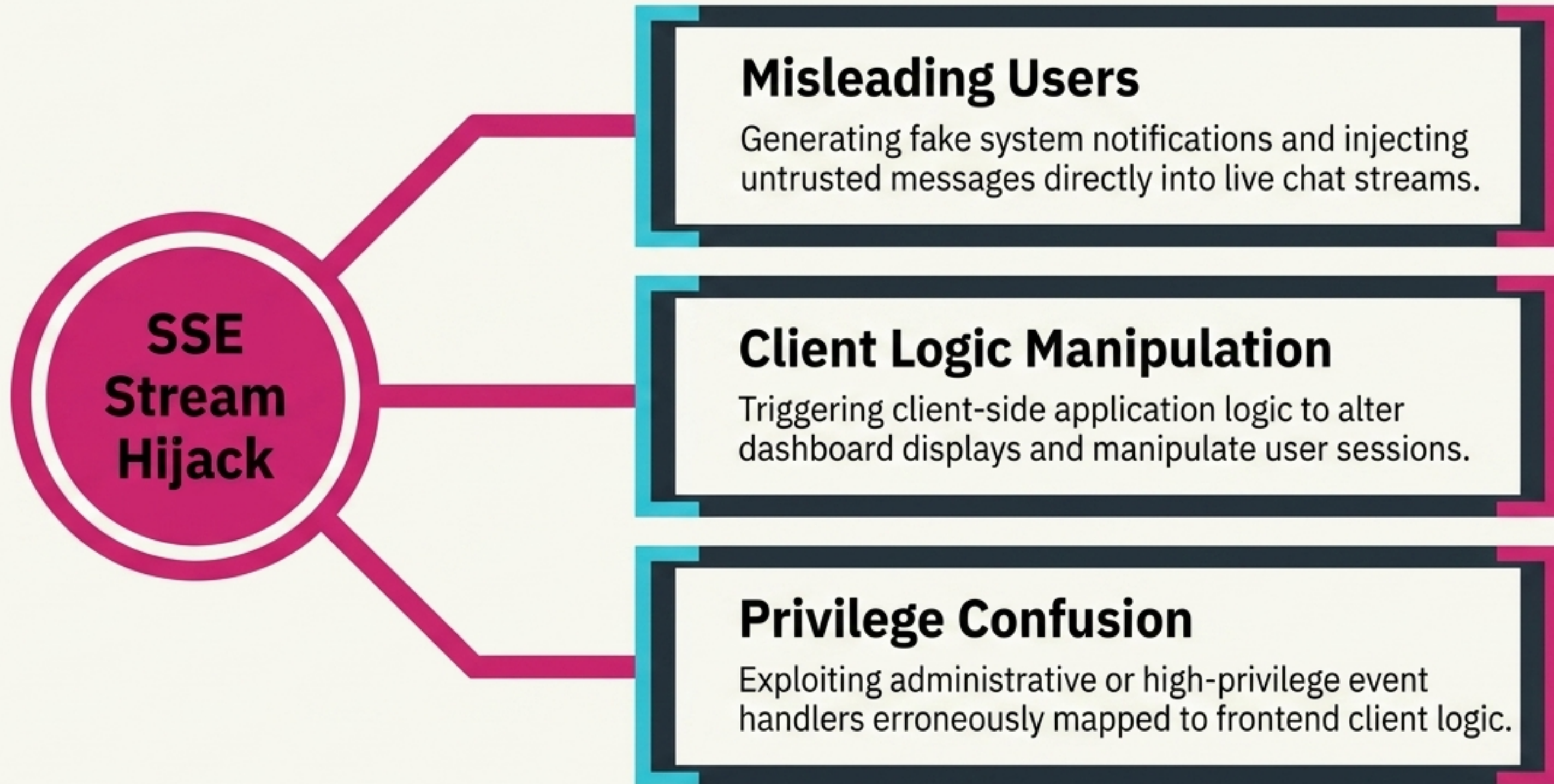
Availability: NONE

No service disruption.

CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:C/C:L/I:H/A:N

Overall Score: 7.5 (HIGH)

Structural stream manipulation escalates rapidly into business logic abuse.



Adversaries leverage stream pollution for data manipulation and masquerading.

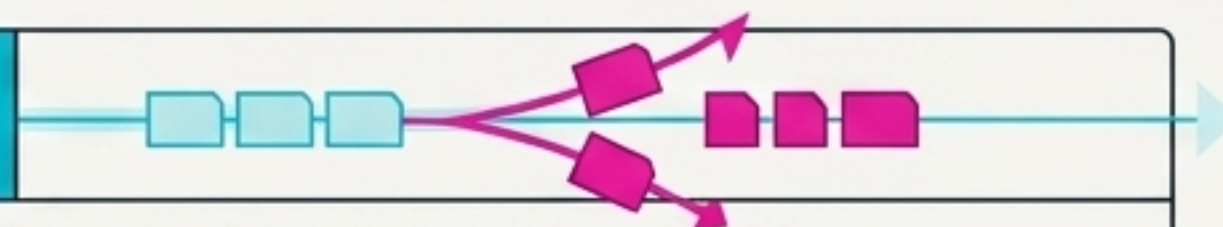
T1190



Exploit Public-Facing Application

The entry point. Attackers exploit the unauthenticated network input in the h3 framework to deliver the payload.

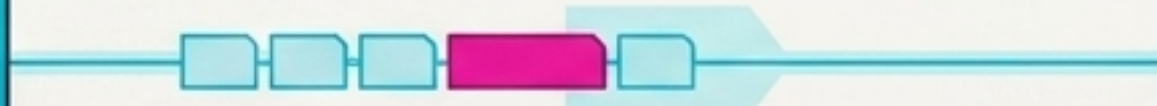
T1565



Data Manipulation

The core mechanism. Using CRLF injections to alter the structural integrity of the outgoing server data.

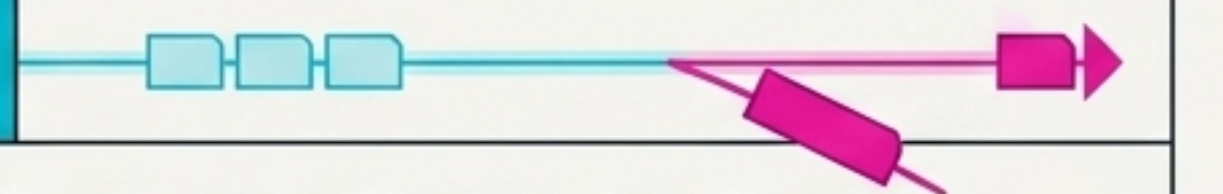
T1036



Masquerading

Disguising the injected malicious payload as a legitimate, server-generated event authenticated by the protocol.

T1059



Client-Side Script Manipulation

The ultimate goal. Altering browser behavior and forcing the client to execute the fabricated commands.

Hunting the flaw requires monitoring for structural anomalies in both inputs and events.

Log Anomalies



Search for raw CRLF characters (`\r`, `\n`) in user-controlled input fields.



Flag instances of multi-line user input where single-line strings are explicitly expected.



Identify unexpected or malformed event structures in web server logs.

Active Defenses



Configure WAF rules to alert on stream boundary injections within SSE endpoints.



Monitor application logs for unknown or unexpected SSE Event IDs.



Track browser event anomalies and unexpected client notification triggers.

Secure the stream by ensuring data can never become structure.

Immediate Fixes

Upgrade to h3 1.15.6+

Upgrade to h3 2.0.1-rc.15+

Framework updates natively resolve the missing newline sanitization oversight.

Security Controls Checklist



Sanitize all newline characters before processing stream data.



Strictly validate all SSE fields (id, event, data, comment) against expected formats.



Encode untrusted input to neutralize special characters.



Review all existing streaming implementations for similar logic gaps.